

Parte 4. Conceptos

1. Diferencia entre Git y GitHub

Pregunta: Explica la diferencia entre Git y GitHub.

Respuesta: Git es un **sistema de control de versiones**: un programa que se instala localmente en la computadora y que permite registrar el historial de cambios de un proyecto, crear ramas, comparar versiones y combinar trabajo de distintas personas, funcionando incluso sin conexión a internet. GitHub, en cambio, es una **plataforma en la nube** que aloja repositorios de Git y agrega funciones de colaboración alrededor de Git, como Pull Requests, revisión de código, control de incidencias (Issues), automatización (Actions) y gestión de accesos de equipos.

Explicación: Git es la herramienta (el motor) que controla las versiones; GitHub es un servicio que utiliza Git como base pero le añade una interfaz web y herramientas de colaboración en equipo. Es un error común pensar que son lo mismo: se puede usar Git perfectamente sin usar nunca GitHub (por ejemplo, guardando el repositorio en un servidor propio), y existen además otras plataformas equivalentes a GitHub que también se basan en Git, como GitLab o Bitbucket.

2. Para qué sirve .gitignore

Pregunta: Explica para qué sirve .gitignore.

Respuesta: El archivo `.gitignore` sirve para indicarle a Git qué archivos o carpetas **no debe rastrear ni incluir** en el control de versiones, es decir, cuáles debe ignorar aunque existan dentro de la carpeta del proyecto.

Explicación: No todos los archivos que existen en la carpeta de un proyecto deben formar parte del historial de Git. Hay archivos que son generados automáticamente (como el entorno virtual `.venv/` o las carpetas `__pycache__`), archivos temporales del sistema operativo o del editor, y archivos que contienen información sensible (como contraseñas o llaves en un archivo `.env`). Al listarlos en `.gitignore`, se evita subirlos por error a GitHub, se mantiene el repositorio más ligero y organizado, y se reduce el riesgo de filtrar información privada.

3. Por qué .venv no debe almacenarse normalmente en GitHub

Pregunta: Explica por qué .venv no debe almacenarse normalmente en GitHub.

Respuesta: `.venv` es la carpeta del **entorno virtual** de Python, y contiene una copia completa del intérprete de Python junto con todas las librerías instaladas para ese proyecto específico. No debe subirse a GitHub porque es un contenido **generado, pesado y dependiente de la máquina** donde fue creado (contiene rutas absolutas y binarios ligados al sistema operativo).

Explicación: En lugar de compartir el entorno virtual ya armado, lo que se comparte es el archivo `requirements.txt`, que enumera únicamente los nombres y versiones de las librerías necesarias. Cualquier persona que descargue el proyecto puede crear su propio `.venv` localmente y ejecutar `pip install -r requirements.txt` para reconstruir un entorno equivalente, funcional en su propio sistema operativo. Subir `.venv` a GitHub aumentaría enormemente el tamaño del repositorio sin aportar ningún beneficio real, y por eso se agrega a `.gitignore`.

4. Para qué sirve requirements.txt

Pregunta: Explica para qué sirve requirements.txt.

Respuesta: `requirements.txt` es un archivo de texto que enumera las **dependencias** (librerías externas) que necesita el proyecto para funcionar, generalmente incluyendo el nombre de cada paquete junto con la versión exacta utilizada.

Explicación: Su propósito es hacer que el entorno del proyecto sea **reproducible**: cualquier otra persona (un compañero de equipo, un servidor de despliegue, o incluso yo mismo en otra computadora) puede instalar exactamente las mismas librerías, en las mismas versiones, con un solo comando (`pip install -r requirements.txt`), evitando así errores causados por diferencias de versiones entre distintos entornos de trabajo.

5. Diferencia entre Stage, Commit y Push

Pregunta: Explica la diferencia entre Stage, Commit y Push.

Respuesta:

- **Stage** (`git add`) - Selecciona qué cambios, de todos los que existen en el directorio de trabajo, quiero incluir en el próximo commit. Los coloca en el área de preparación, pero todavía no quedan registrados en el historial.

- **Commit** (`git commit`) - Toma lo que está en el área de preparación y lo guarda de forma permanente como una nueva versión en el **historial local** del repositorio, junto con un mensaje descriptivo. Sigue siendo una operación local.

- **Push** (`git push`) - Envía los commits que tengo localmente hacia el **repositorio remoto** (por ejemplo, en GitHub), haciendo que esos cambios sean visibles y accesibles para otras personas.

Explicación: Las tres operaciones representan niveles progresivos de "confirmación" de un cambio: primero decido qué cambios importan (Stage), después los registro de forma definitiva en mi historial local (Commit), y finalmente decido compartir ese historial con los demás (Push). Mantenerlas separadas da control: puedo hacer varios commits pequeños y organizados localmente antes de decidir, en un solo momento, compartir todo ese trabajo.

6. Por qué un repositorio puede tener varios commits antes de realizar un push

Pregunta: Explica por qué un repositorio puede tener varios commits antes de realizar un push.

Respuesta: Porque **commit** es una operación completamente **local**: cada vez que hago un commit, únicamente estoy guardando una versión en el historial de mi propia computadora, sin que eso implique ninguna comunicación con el repositorio remoto. **Push**, en cambio, es una acción separada y deliberada que decido ejecutar cuando quiero sincronizar ese historial local con GitHub.

Explicación: Esto le da flexibilidad al flujo de trabajo: puedo ir guardando mi progreso en pequeños commits frecuentes y bien organizados (por ejemplo, uno por cada pequeña tarea completada) mientras trabajo, incluso sin conexión a internet, y decidir después -cuando el trabajo esté listo o al final de una sesión- subir todos esos commits acumulados de una sola vez con un solo `git push`. No es necesario, ni obligatorio, hacer push después de cada commit individual.